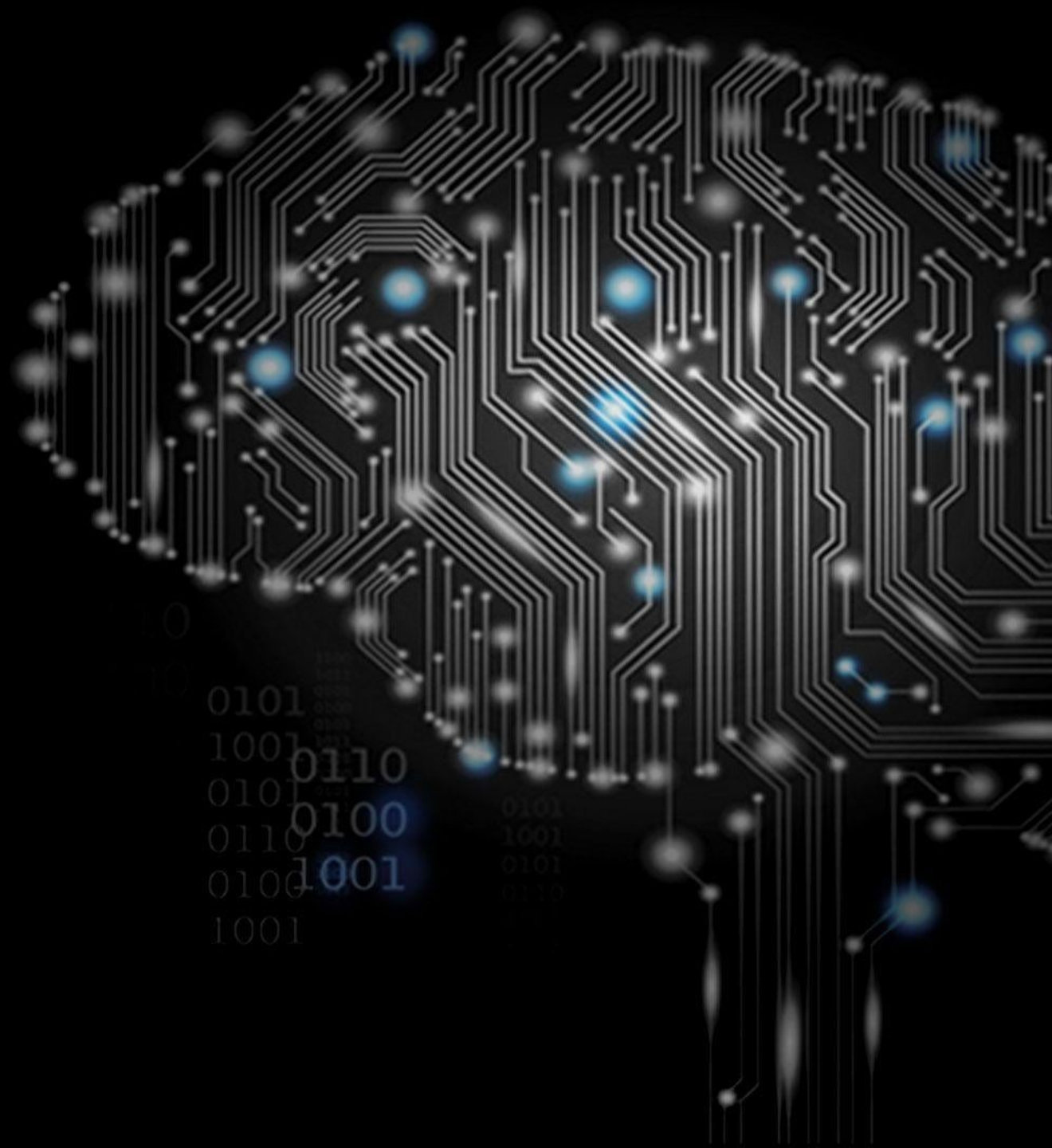


Introduction to Generative AI

3. More Prompt Engineering



Prompting Techniques

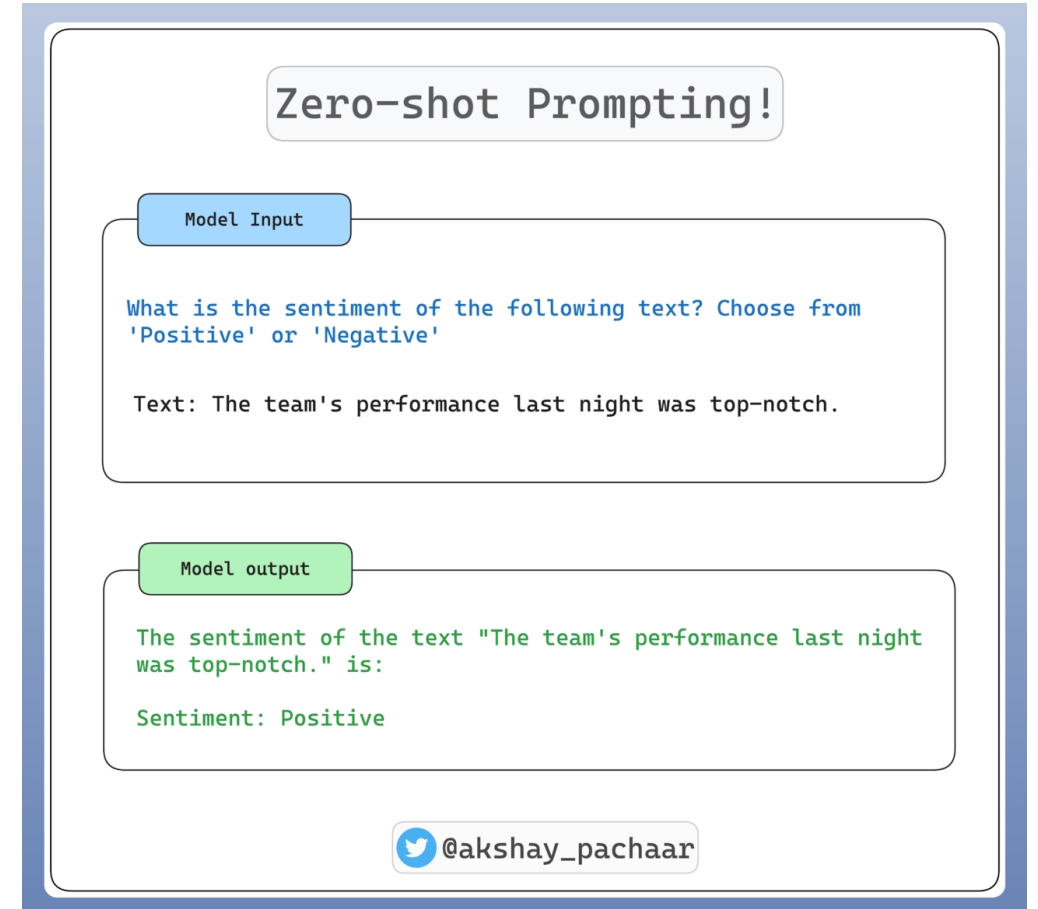
- Modern prompting techniques exploit how LLMs learn from context
- They are structured ways of shaping probability distributions



Prompting Techniques

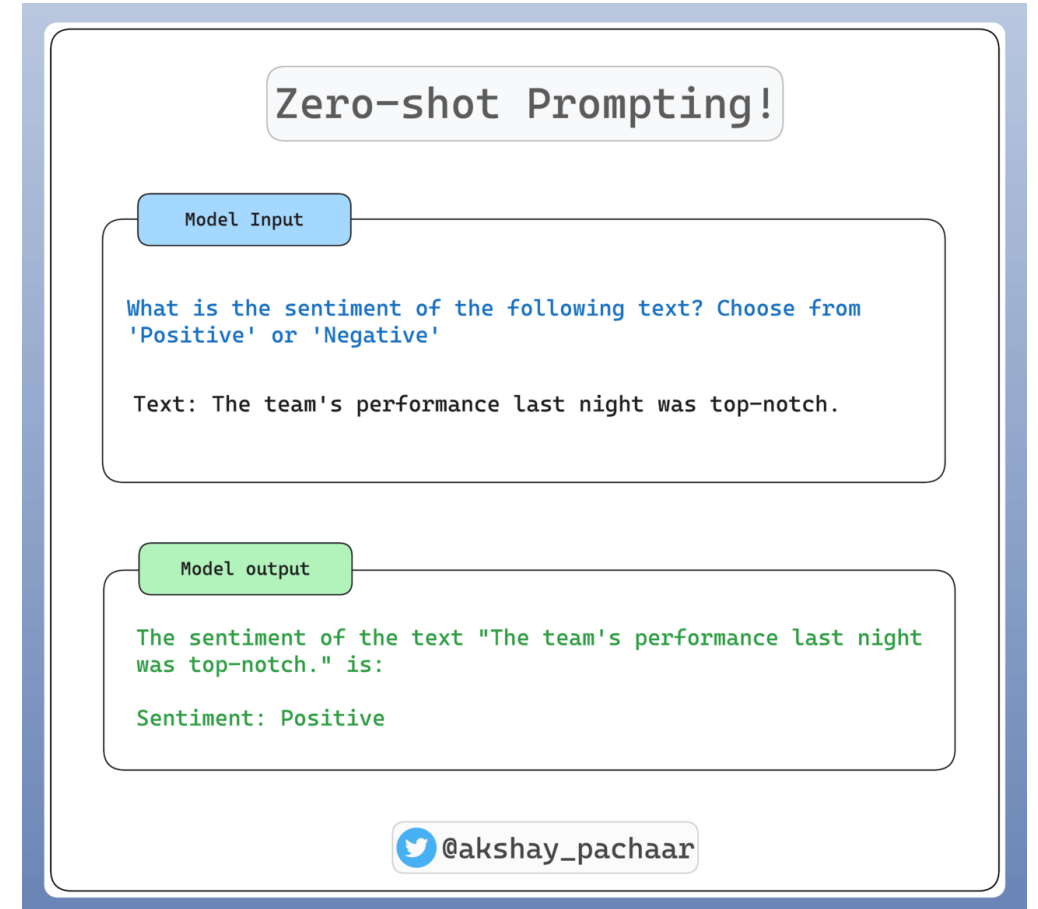
- Zero-shot prompting
 - Asking the model to perform a task without providing any examples in the prompt
 - Relies entirely on the model's pretraining and instruction tuning
 - Zero-shot prompting often improves when a role is defined
 - *Narrows reasoning space*
 - *Increases domain alignment*
 - *Reduces generic output*

You are a financial risk analyst.
Assess the following investment scenario.
Provide 3 risk factors.



Prompting Techniques

- Zero-shot prompting
 - Works best for
 - *Common tasks (summarization, classification)*
 - *Clearly defined output formats*
 - *Well-known domains*
 - Limitations
 - *Higher ambiguity*
 - *Less control over formatting*
 - *More variance in reasoning*



Prompting Techniques

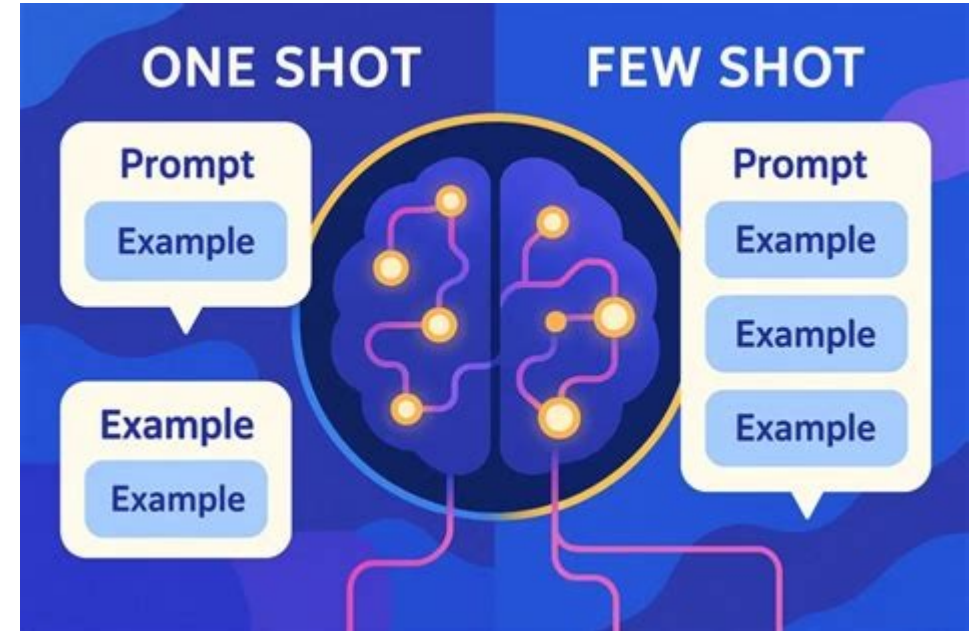
- One-Shot and Few-Shot prompting
 - Few-shot prompting provides examples inside the prompt to demonstrate the desired pattern
 - This leverages in-context learning
 - Instead of only telling the model what to do, it is shown what to do

```
Classify the sentiment as Positive, Neutral, or Negative.
```

```
Input: I love this phone.  
Output: Positive
```

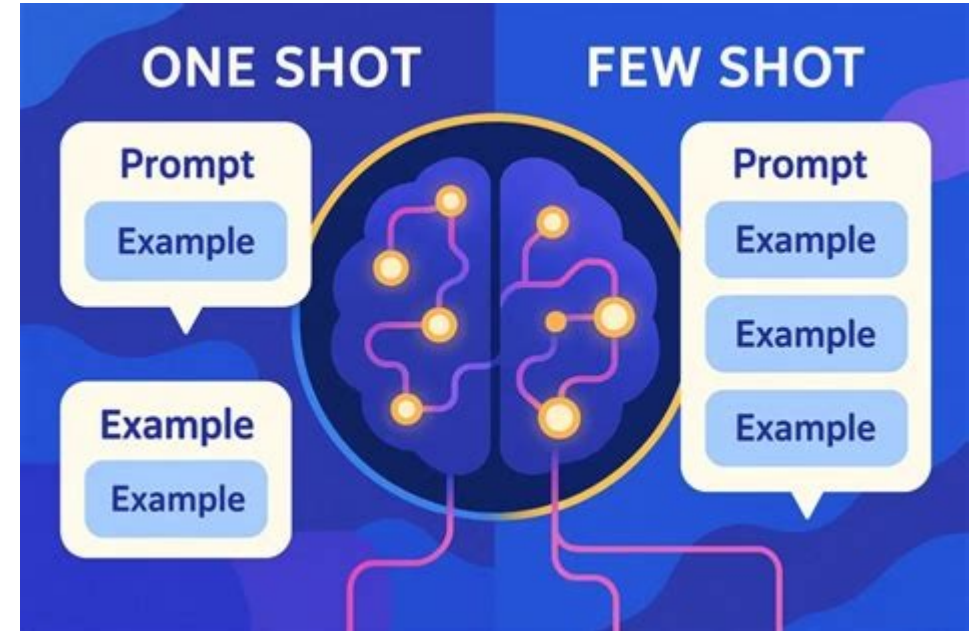
```
Input: It works fine.  
Output: Neutral
```

```
Input: This is terrible.  
Output:
```



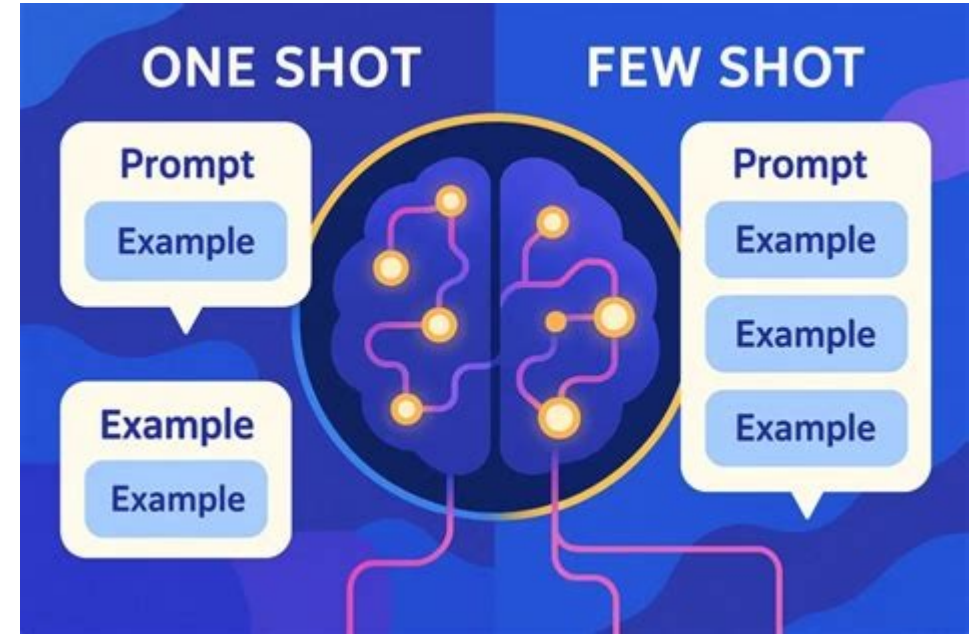
Prompting Techniques

- Why few-shot works
 - LLMs are trained to complete patterns
 - Providing examples
 - *Reduces ambiguity*
 - *Reduces format drift*
 - *Improves consistency*
 - Often increases accuracy on niche tasks
 - The model infers
 - *Pattern structure*
 - *Output format*
 - *Task logic*



Prompting Techniques

- When to use few-shot
 - Complex output structure
 - Custom classification schemes
 - Non-standard tasks
 - Domain-specific labeling
 - Consistency is critical
 - Inconsistent examples reduce reliability
- Few-shot increases
 - Token usage
 - Cost
 - Latency
 - It also reduces available context window



Prompting Techniques

- Chain-of-thought (CoT) prompting
 - Encourages the model to produce intermediate reasoning steps before the final answer

Solve the following step by step and explain your reasoning:

What is 17×24 ?

Let's multiply 17×24 step by step.

1. Break 17 into $10 + 7$

$$17 \times 24 = (10 + 7) \times 24$$

2. Distribute (multiply each part by 24)

$$= 10 \times 24 + 7 \times 24$$

3. Compute each product

- $10 \times 24 = 240$
- $7 \times 24 = 168$ (since $7 \times 20 = 140$ and $7 \times 4 = 28$, total $140 + 28 = 168$)

4. Add them together

$$240 + 168 = 408$$

✓ Answer: $17 \times 24 = 408$

Standard Prompting

Input

Q: Roger has 5 tennis balls. He buys 2 more cans of tennis balls. Each can has 3 tennis balls. How many tennis balls does he have now?

A: The answer is 11.

Q: The cafeteria had 23 apples. If they used 20 to make lunch and bought 6 more, how many apples do they have?

Model Output

A: The answer is 27. ✗

Chain of Thought Prompting

Input

Q: Roger has 5 tennis balls. He buys 2 more cans of tennis balls. Each can has 3 tennis balls. How many tennis balls does he have now?

A: Roger started with 5 balls. 2 cans of 3 tennis balls each is 6 tennis balls. $5 + 6 = 11$. The answer is 11.

Q: The cafeteria had 23 apples. If they used 20 to make lunch and bought 6 more, how many apples do they have?

Model Output

A: The cafeteria had 23 apples originally. They used 20 to make lunch. So they had $23 - 20 = 3$. They bought 6 more apples, so they have $3 + 6 = 9$. The answer is 9. ✓

Prompting Techniques

- Why CoT improves accuracy
 - LLMs sometimes fail at Implicit multi-step reasoning
 - When forced to
 - *Externalize reasoning steps*
 - *Expand intermediate calculations*
 - They reduce error accumulation
 - This works because
 - *The model is better at short reasoning steps than large ones*

Standard Prompting

Input

Q: Roger has 5 tennis balls. He buys 2 more cans of tennis balls. Each can has 3 tennis balls. How many tennis balls does he have now?

A: The answer is 11.

Q: The cafeteria had 23 apples. If they used 20 to make lunch and bought 6 more, how many apples do they have?

Model Output

A: The answer is 27. ❌

Chain of Thought Prompting

Input

Q: Roger has 5 tennis balls. He buys 2 more cans of tennis balls. Each can has 3 tennis balls. How many tennis balls does he have now?

A: Roger started with 5 balls. 2 cans of 3 tennis balls each is 6 tennis balls. $5 + 6 = 11$. The answer is 11.

Q: The cafeteria had 23 apples. If they used 20 to make lunch and bought 6 more, how many apples do they have?

Model Output

A: The cafeteria had 23 apples originally. They used 20 to make lunch. So they had $23 - 20 = 3$. They bought 6 more apples, so they have $3 + 6 = 9$. The answer is 9. ✅



Prompting Techniques

- When it is useful
 - Math word problems
 - Logical deduction
 - Planning
 - Code reasoning
 - Multi-step transformations
- Chain-of-thought has downsides
 - Longer outputs incur higher cost
 - Increased hallucination surface
 - Possible exposure of flawed reasoning
 - Not always necessary for simple tasks

Standard Prompting

Input

Q: Roger has 5 tennis balls. He buys 2 more cans of tennis balls. Each can has 3 tennis balls. How many tennis balls does he have now?

A: The answer is 11.

Q: The cafeteria had 23 apples. If they used 20 to make lunch and bought 6 more, how many apples do they have?

Model Output

A: The answer is 27. ❌

Chain of Thought Prompting

Input

Q: Roger has 5 tennis balls. He buys 2 more cans of tennis balls. Each can has 3 tennis balls. How many tennis balls does he have now?

A: Roger started with 5 balls. 2 cans of 3 tennis balls each is 6 tennis balls. $5 + 6 = 11$. The answer is 11.

Q: The cafeteria had 23 apples. If they used 20 to make lunch and bought 6 more, how many apples do they have?

Model Output

A: The cafeteria had 23 apples originally. They used 20 to make lunch. So they had $23 - 20 = 3$. They bought 6 more apples, so they have $3 + 6 = 9$. The answer is 9. ✅



Prompting Techniques

- Decomposition prompting
 - Breaks a complex task into smaller sub-tasks
 - Instead of “Design a distributed database architecture”
- You break it into
 - Define system requirements
 - Choose consistency model
 - Design replication strategy
 - Define scaling approach



Prompting Techniques

- Why decomposition works
 - LLMs struggle with:
 - *Large cognitive jumps*
 - *High-entropy tasks*
 - *Complex planning*
- Breaking down tasks reduces
 - Cognitive load per step
 - Error compounding
 - Logical drift



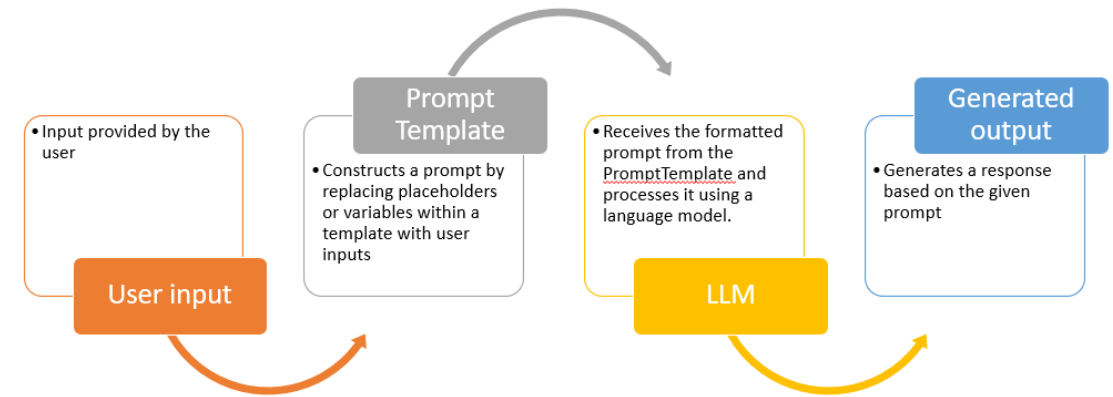
Prompting Techniques

- Iterative prompting
 - In production systems, decomposition often happens across multiple API calls
 - *Generate outline*
 - *Review outline*
 - *Expand section 1*
 - *Validate output*
 - *Continue*
 - This improves
 - *Reliability*
 - *Control*
 - Error detection



Prompt Templates

- In a production system
 - Prompts are development artifacts
 - Treated like code artifacts or test artifacts
- They should be
 - Modular
 - Reusable
 - Versioned
 - Testable



Prompt Templates

- Without templates
 - Prompts become inconsistent
 - Behavior drifts
 - Debugging becomes difficult
 - Updates cause regressions
 - Security risks increase
- With templates
 - Outputs are predictable
 - Prompts are reusable
 - Versioning becomes possible
 - Testing is easier
 - Scaling is manageable
 - Consistent with software engineering

```
system = """You are a {role}.
```

```
Follow these constraints:
```

```
{constraints}
```

```
"""
```

```
user = """<task>
```

```
{task_description}
```

```
</task>
```

```
<input>
```

```
{input_data}
```

```
</input>
```

```
<output_format>
```

```
{output_format}
```

```
</output_format>
```

```
"""
```



Prompt Templates

- Sample classification template

```
System: You are a classification assistant.
```

```
User:
```

```
<task>
```

```
Classify the following text into one of these categories
```

```
- Billing
```

```
- Technical Support
```

```
- Sales
```

```
- Other
```

```
</task>
```

```
<text>
```

```
{{customer_message}}
```

```
</text>
```

```
<output>
```

```
Return only the category name. No explanation.
```

```
</output>
```



Prompt Templates

- Summarization template

```
System: You are an executive summarization assistant.
```

```
User:
```

```
<task>
```

```
Summarize the following document in {{num_sentences}} sentences.
```

```
</task>
```

```
<document>
```

```
{{document_text}}
```

```
</document>
```

```
<constraints>
```

- Focus on key business insights
- Avoid minor details
- Do not add information

```
</constraints>
```

```
<output>
```

```
Plain text summary only.
```

```
</output>
```



Prompt Templates

- Code generation template

```
System: You are a senior Python engineer.
```

```
User:
```

```
<task>
```

```
Write a function that satisfies the following specification.
```

```
</task>
```

```
<specification>
```

```
Function Name: {{function_name}}
```

```
Input: {{input_description}}
```

```
Output: {{output_description}}
```

```
</specification>
```

```
<constraints>
```

```
- Python 3.11
```

```
- No external libraries
```

```
- Include type hints
```

```
- Handle edge cases
```

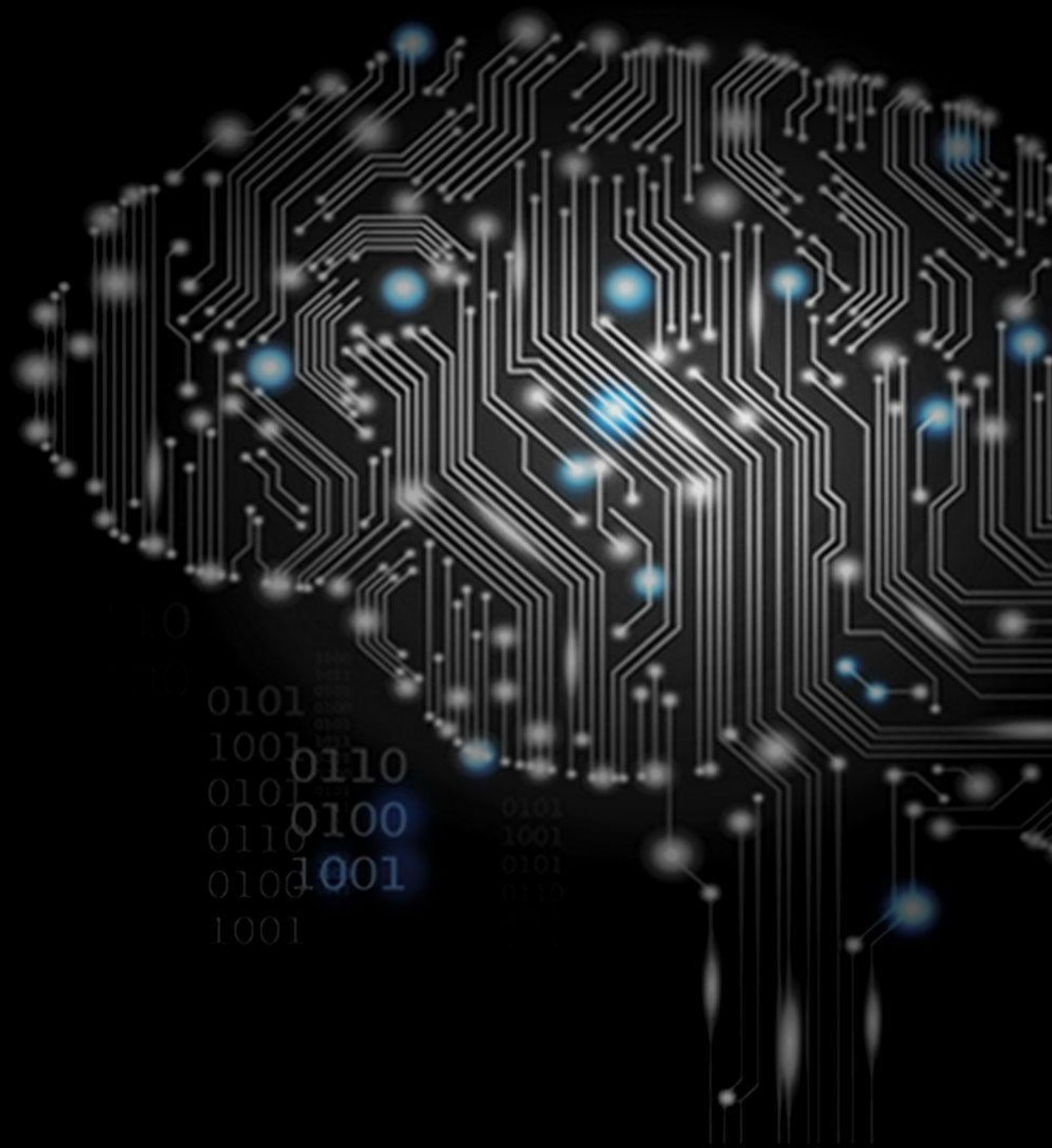
```
</constraints>
```

```
<output>
```

```
Return only the code. No prose.
```

```
</output>
```





0

0101
1001
0101
0110
0100
1001
1001

0101
1001
0101
0110